

Chapter 9: Other Universal Agents

An Introduction to Universal Artificial Intelligence

Optimism, Sampling, Knowledge-Seeking, Exploration, and Planning-Avoidance

Marcus Hutter, Elliot Catt, David Quarel

Taylor & Francis / CRC Press, 2024

Chapter Overview

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 Summary

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 Summary

Beyond AIXI

In earlier chapters we met **AIXI** and the Bayes-optimal agent $AI\xi$ (AI-xi), which picks the action maximizing expected value under a Bayesian mixture ξ (xi) over all environments in a model class. AIXI is remarkably general, but it has a serious theoretical weakness: it fails to be *asymptotically optimal* – there exist environments in which AIXI never learns to act as well as the best achievable policy, no matter how much data it sees (this was discussed in Section 8.3).

This chapter surveys **five extensions** of AIXI that each try to fix this or a related shortcoming. Two of them (BayesExp and Inq) actually achieve asymptotic optimality.

- **9.1 Optimistic agents:** instead of averaging over environments, act as if the *best-case* environment consistent with the data were true.
- **9.2 (Thompson) sampling agents:** act optimally for one environment *sampled* from the current posterior.
- **9.3 Knowledge-seeking agents:** replace the reward signal entirely with a drive to maximize information gained about the environment.
- **9.4 Exploring agents (BayesExp, Inq):** occasionally force deliberate exploration on top of Bayes-optimal behaviour.
- **9.5 Planning-avoiding agents (Self-AIXI):** replace expensive expectimax/Monte-Carlo-Tree-Search planning with self-predicted, one-step Q-value maximization.

Notation Recap

Throughout this chapter we reuse notation from earlier chapters. As a quick reminder before we dive in:

- $h_{<t} = a_1 e_1 a_2 e_2 \cdots a_{t-1} e_{t-1}$ is the **history** of actions a and percepts (observation/reward pairs) e observed before time t .
- π (pi) denotes a **policy**, a (possibly stochastic) function from histories to actions; Π (capital pi) is a class of policies.
- ν (nu) denotes a candidate **environment** (a stochastic function from history and action to the next percept); μ (mu) denotes the one true, but unknown, environment; $\mathcal{M}$ is a **class** (set) of candidate environments.
- w_ν is the **prior weight** the agent assigns to environment ν before seeing any data, and $w(\nu|h_{<t})$ is the **Bayesian posterior** weight after observing history $h_{<t}$.
- $\xi(\cdot|h_{<t}) := \sum_{\nu \in \mathcal{M}} w_\nu \nu(\cdot|h_{<t})$ is the **Bayes mixture**: a single environment built by averaging all candidates, weighted by how much the agent currently believes in each of them.
- $V_\nu^\pi(h_{<t})$ is the **value** (expected discounted future reward) of following policy π in environment ν having observed $h_{<t}$; γ (gamma) is the discount factor and $\Gamma_t = \sum_{k \geq t} \gamma^k$ normalizes discounted sums.
- $\text{AI}\xi$ denotes the Bayes-optimal agent $\pi_\xi^*(h_{<t}) := \arg \max_\pi V_\xi^\pi(h_{<t})$: the policy that maximizes expected value under the mixture ξ .

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents**
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 Summary

Optimism vs. Pessimism

For *sequence prediction*, it is known that **pessimism** (planning for the worst case) can be an optimal strategy. Surprisingly, in the history-based **reinforcement learning** setting, acting *optimistically* instead leads to optimal behaviour.

What do optimism and pessimism mean here? The agent assumes itself to be interacting with the *best* (optimism) or *worst* (pessimism) possible environment – where “best”/“worst” means the environment ν with the highest/lowest value V_ν^* , restricted to those environments in the model class $\mathcal{M}$ that remain **consistent** with the history observed so far.

The Optimistic Policy π°

$$\pi^\circ(h_{<t}) := \arg \max_{\pi} \max_{\nu \in \mathcal{M}_t} V_\nu^\pi(h_{<t})$$

Plain English. π° (“pi-circle”, the optimistic policy) jointly searches over every policy π *and* every environment ν still consistent with the data ($\mathcal{M}_t$, updated at every time step by discarding ruled-out environments), and commits to whichever policy achieves the single highest value against its best-case environment.

Optimism vs. Bayes-Averaging

Compare: the Bayes-Optimal Policy

$$\pi_{\xi}^*(h_{<t}) := \arg \max_{\pi} V_{\xi}^{\pi}(h_{<t}) := \arg \max_{\pi} \sum_{\nu \in \mathcal{M}} V_{\nu}^{\pi}(h_{<t})$$

Plain English. AI_{ξ} instead *averages* the value over all environments (weighted by the mixture ξ), rather than taking the best case. Optimism replaces the sum with a max.

Algorithm 9.1: Optimistic Agent for Deterministic Environments

[SH12a] developed an optimistic agent for a **finite class of deterministic environments** $\mathcal{M}_0$. The algorithm finds the most optimistic (policy, environment) pair, follows that policy, and at each step removes environments inconsistent with the observed history. If the chosen environment ν^* is ever found inconsistent, a new optimistic pair is selected.

Require: Environment class $\mathcal{M}_0 = \{\nu_1, \dots, \nu_m\}$. Some deterministic policy class Π

Input: Value function V_ν^π . True environment μ

Output: Interaction history $h = a_1 e_1 a_2 e_2 a_3 e_3 \dots$

```
1:  $t := 1$ 
2: while  $\mathcal{M}_{t-1} \neq \{\}$  do
3:    $(\pi^*, \nu^*) := \arg \max_{\pi \in \Pi, \nu \in \mathcal{M}_{t-1}} V_\nu^\pi(h_{<t})$ 
4:   while  $\nu^* \in \mathcal{M}_{t-1}$  do                                     ▷ inner loop is an optional speed-up
5:      $a_t := \pi^*(h_{<t})$ 
6:     Perceive  $e_t$  from environment  $\mu$ 
7:      $h_{1:t} := h_{<t} a_t e_t$ 
8:      $\mathcal{M}_t := \{\nu \in \mathcal{M}_{t-1} : \nu(h_{<t} a_t) = e_t\}$            ▷ remove all inconsistent  $\nu$ 
9:      $t := t + 1$ 
10:  end while
11: end while
```

Algorithm 9.1: Plain-English Summary

Plain English. At every cycle the agent picks the single (policy, environment) pair with the largest value, acts according to that policy, and keeps only the environments whose predictions matched what actually happened. Because environments here are deterministic, a single mismatched percept e_t is enough to permanently discard ν .

Theorem 9.1.1: Optimism Is Asymptotically Optimal (Deterministic)

Theorem 9.1.1 (Optimism is asymptotically optimal for finite deterministic classes [SH12a])

Suppose $\mathcal{M}$ is a finite class of deterministic environments. Suppose Π is any class of deterministic policies. If we use Algorithm 9.1 (π°) in an environment $\mu \in \mathcal{M}$, then there is a $T < \infty$ such that

$$V_\mu^{\pi^\circ}(h_{<t}) = \sup_{\pi \in \Pi} V_\mu^\pi(h_{<t}) \quad \forall t \geq T$$

Plain English. After some finite time T , the optimistic agent's actual value *exactly equals* the best possible value in the true environment μ – not just approximately, but exactly, for every later time step.

Proof sketch. Since $\mathcal{M}_0$ is finite, there is a time $T < \infty$ by which every environment that is *more optimistic* than the truth μ has been ruled out. The most optimistic environment ν that is never ruled out must in fact generate the exact same on-policy percepts as μ forever after – otherwise it would eventually be refuted. Hence its optimal policy π_ν^* coincides with π_μ^* , giving the result. ■

These results hold for any deterministic policy class Π , and (since the resulting sequence of optimal policies is deterministic anyway) they also hold when Π is our usual, larger class of *all stochastic* policies.

Theorem 9.1.2: Finite Error Bound I

Theorem 9.1.2 (Finite error bound [SH12a])

For geometric discount $\gamma_t = \gamma^t$ with constant $\gamma \in (0, 1)$, and $0 < \varepsilon < 1$, following policy π° of Algorithm 9.1, except for at most $|\mathcal{M}| \cdot \left\lceil \frac{\log \varepsilon}{\log \gamma} \right\rceil$ time steps t , its value is ε -optimal:

$$V_\mu^{\pi^\circ}(h_{<t}) \geq \max_{\pi \in \Pi} V_\mu^\pi(h_{<t}) - \varepsilon$$

Plain English. ε (epsilon) is a small tolerance we choose. The theorem says the optimistic agent's value is within ε of the best possible value at *all but finitely many* time steps, and it even bounds exactly how many “bad” steps there can be: at most $|\mathcal{M}| \lceil \log \varepsilon / \log \gamma \rceil$ of them (roughly $|\mathcal{M}| \cdot \lceil -\ln \varepsilon / (1 - \gamma) \rceil$ when γ is close to 1). Since this holds for *every* $\varepsilon > 0$, taking $\Pi = \text{All}$ implies π° is asymptotically optimal.

Theorem 9.1.2: Finite Error Bound II

Proof. For the ℓ -truncated value $V_{\nu}^{\pi, t+\ell}(h_{<t})$ (which only looks ℓ steps into the future instead of infinitely far),

$$|V_{\nu}^{\pi, t+\ell}(h_{<t}) - V_{\nu}^{\pi}(h_{<t})| \leq \frac{1}{\Gamma_t} \sum_{k=t+\ell+1}^{\infty} \gamma^k = \gamma^{\ell+1} \leq \varepsilon$$

for $\ell + 1 := \lceil \log \varepsilon / \log \gamma \rceil$ (positive because both $\log \varepsilon$ and $\log \gamma$ are negative).

Let (π_t^*, ν_t^*) be the (policy, environment) pair Algorithm 9.1 selects in cycle t . Suppose first that ν_t^* stays consistent with the actual history for the next ℓ steps – then π_t^*, ν_t^* do not change over that window (inner loop of Algorithm 9.1), and a chain of (in)equalities gives

$$V_{\mu}^{\pi^{\circ}}(h_{<t}) \geq \max_{\pi} \max_{\nu \in \mathcal{M}_t} V_{\nu}^{\pi}(h_{<t}) - \gamma^{\ell+1} \geq \max_{\pi \in \Pi} V_{\mu}^{\pi}(h_{<t}) - \varepsilon,$$

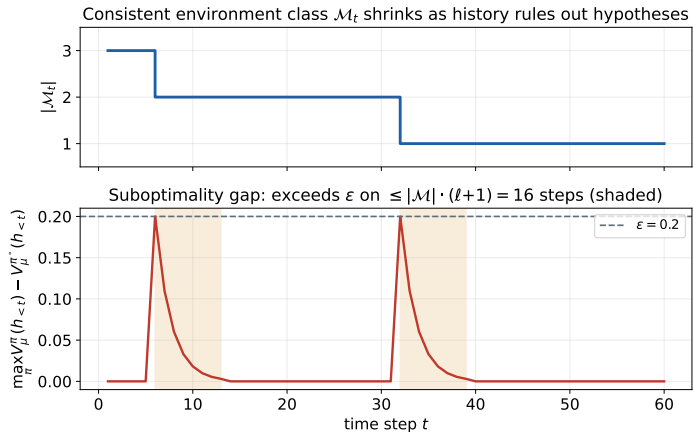
using truncation, the fact that π° agrees with π_t^* on this window, the definition of (π_t^*, ν_t^*) as the optimistic maximizer, and finally that $\mu \in \mathcal{M}_t$ is one of the candidates being maximized over.

Theorem 9.1.2: Finite Error Bound III

Now let $t_1, \dots, t_K$ be exactly the time steps at which the *currently selected* ν_t^* becomes inconsistent with the true history. The bound above can fail only in a window of $\ell + 1$ steps around each such t_i , i.e. for $t \in \mathcal{T}_\times := \bigcup_{i=1}^K \{t_i - \ell, \dots, t_i\}$. Since there are at most $|\mathcal{M}| - 1$ possible refutation events (each removes at least one environment, and μ itself is never removed), $K \leq |\mathcal{M}| - 1$, so

$$|\mathcal{T}_\times| = (\ell + 1)K \leq |\mathcal{M} - 1| \left\lceil \frac{\log \varepsilon}{\log \gamma} \right\rceil. \quad \blacksquare$$

Theorem 9.1.2: Finite Error Bound IV



Theorem 9.1.2: Finite Error Bound V

Illustration. Top: $\mathcal{M}_t$ shrinks each time an environment is refuted ($3 \rightarrow 2 \rightarrow 1$). Bottom: the suboptimality gap exceeds ε only during the finitely many shaded windows right after a refutation. (Synthetic example: $|\mathcal{M}_0| = 3$, $\gamma = 0.8$, $\varepsilon = 0.2$, $\ell + 1 = 8$, bound of 16 bad steps.)

Algorithm 9.2: Optimistic Agent for Stochastic Environments

A similar algorithm handles a **stochastic** environment class $\mathcal{M}_0$. The key difference from Algorithm 9.1 is *how* inconsistent environments are removed: a stochastic environment can never be refuted with absolute certainty, so we discard any ν whose likelihood of the observed history falls far below (by a factor z) that of the best-fitting candidate.

Require: $\mathcal{M}_0 = \{\nu_1, \dots, \nu_m\}$. Deterministic policy class Π

Require: Threshold $z \in (0, 1)$

Input: Value function V_ν^π . True environment μ

Output: Interaction history $h = a_1 e_1 a_2 e_2 a_3 e_3 \dots$

1: $t := 1$

2: **while** True **do**

3: $(\pi^*, \nu^*) \in \arg \max_{\pi \in \Pi, \nu \in \mathcal{M}_{t-1}} V_\nu^\pi(h_{<t})$

4: $a_t := \pi^*(h_{<t})$

5: Sample $e_t \sim \mu(\cdot | h_{<t} a_t)$

6: $h_{1:t} := h_{<t} a_t e_t; \quad t := t + 1$

7: $\mathcal{M}_t := \left\{ \nu \in \mathcal{M}_{t-1} : \frac{\prod_{j=1}^t \nu(e_j | h_{<j} a_j)}{\max_{\tilde{\nu} \in \mathcal{M}_0} \prod_{j=1}^t \tilde{\nu}(e_j | h_{<j} a_j)} > z \right\}$

▷ remove inconsistent ν

8: **end while**

Algorithm 9.2: Plain-English Summary

Plain English. An environment ν survives only while the total probability it assigns to everything observed so far stays within a factor z of the best-fitting candidate's probability; falling behind means ν is discarded, even though it was never technically “wrong”.

Theorem 9.1.3: Optimality for Finite Stochastic Classes

Theorem 9.1.3 (Optimality, finite stochastic class [SH12a])

Define π° via Algorithm 9.2 with any threshold $z \in (0, 1)$ and a finite class $\mathcal{M}$ of stochastic environments containing the true environment μ . Then, with probability at least $1 - z|\mathcal{M}|$, there exists for every $\varepsilon > 0$ a time $T < \infty$ such that

$$V_{\mu}^{\pi^\circ}(h_{<t}) > \max_{\pi \in \Pi} V_{\mu}^{\pi}(h_{<t}) - \varepsilon \quad \forall t \geq T$$

Plain English. With high probability (at least $1 - z|\mathcal{M}|$ – close to 1 if the threshold z is small), the optimistic agent again becomes essentially optimal after some finite time, this time up to any desired tolerance ε . The proof uses martingale theory and is found in [SH12a].

Extending Optimism Beyond Finite Classes

Extending the idea. The optimism principle extends further:

- to **countable** classes $\mathcal{M}$, by running the algorithm on a slowly-growing sequence of finite subsets;
- to **compact** classes (w.r.t. a metric $d(\nu, \nu') = \sup_{h, \pi} |V_{\nu}^{\pi}(h) - V_{\nu'}^{\pi}(h)|$), e.g. the class of finite-state (Partially Observable) Markov Decision Processes, by covering $\mathcal{M}$ with finitely many balls of radius $< \varepsilon/2$ and running Algorithm 9.2 on their centres;
- further still to **separable** classes, which can be covered by countably many arbitrarily small balls – but full asymptotic optimality (requiring $\varepsilon \rightarrow 0$) needs the number of ε -errors to stay controlled, which for compact subsets of $\mathbb{R}^d$ is typically *exponential* in d (the curse of dimensionality).
- If $\mathcal{M}$ is generated by finite sets of *partial environments* (“laws of nature”), error bounds linear in the number of laws (rather than the number of environments) are possible – e.g. for (PO)MDPs, exponentially improved bounds polynomial in d exist.

Python: Optimistic Agent on a Toy Deterministic Class

We simulate Algorithm 9.1 on 3 deterministic environments, each defined by the fixed observation sequence it emits; the agent rules them out as mismatches arrive.

```
1  # Fixed observation sequences the 3 candidate environments emit.
2  M0 = {"nu1": "0,1,1,0,1", "nu2": "0,1,0,0,1", "nu3": "0,0,1,0,1"}
3  mu_key = "nu3" # nu3 is the TRUE environment
4
5  M_t = dict(M0)
6  true_obs = M0[mu_key].split(",")
7  for t, e_t in enumerate(true_obs, start=1):
8      survivors = {k: v for k, v in M_t.items()
9                  if v.split(",")[t - 1] == e_t}
10     print(f"t={t} e_t={e_t} |M_t|:{len(M_t)}->{len(survivors)} {list(survivors)}")
11     M_t = survivors
12
13 # t=1 e_t=0 |M_t|:3->3 ['nu1', 'nu2', 'nu3']
14 # t=2 e_t=0 |M_t|:3->1 ['nu3']
15 # t=3 e_t=1 |M_t|:1->1 ['nu3'] (nu3 survives every remaining step)
```

Python Example: What Happened

What happened. At $t = 2$ the true environment emits observation 0, but ν_1 predicted 1 – so ν_1 is discarded immediately. From $t = 2$ onward only ν_3 (the truth) remains consistent, and by Theorem 9.1.1 the agent's policy is then exactly optimal.

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 Summary

The Idea: A Policy That Samples Other Policies

Another way to build a policy that performs well is to have a *policy which samples from other policies*. A successful version of this idea is **Thompson sampling**, originating with [Tho33] for bandit problems and recently rediscovered and applied to reinforcement learning and sequential decision theory.

Like any good Bayesian agent, the Thompson sampler updates its posterior at every iteration. But unlike $AI\xi$, which *mixes* over environments (averages them into ξ), the Thompson agent *samples* a single environment ν from the posterior distribution and follows the policy that is optimal *for that one sampled environment*, for an ε_t -effective horizon, then repeats.

Where is the exploration? It is more subtle than for BayesExp (Section 9.4), which explicitly switches between explore/exploit modes. In one sense Thompson sampling is *always* exploring, since it never commits to the single most likely policy. In another sense, it explores precisely when it happens to sample a policy that is not the most likely one.

Algorithm 9.3: Thompson Sampling Policy

Require: Model class $\mathcal{M}$. Prior w . Effective horizon function H_t .

Input: Percept stream $e_1, e_2, e_3, \dots$

Output: Action stream $a_1, a_2, a_3, \dots$

- 1: **while True do**
- 2: sample environment ν according to the posterior $w(\nu \mid h_{<t})$
- 3: follow π_ν^* for $H_t(\varepsilon_t)$ time steps
- 4: **end while**

Plain English. $H_t(\varepsilon_t)$ is the **effective horizon**: roughly how many future steps still matter given the discount function, for tolerance ε_t . So the recipe is: draw one hypothesis ν about the world from your current beliefs, pretend it is true, act optimally under that pretence for a while, then draw a fresh hypothesis and repeat.

Note the algorithm glosses over *how* to compute the posterior $w(\nu \mid h_{<t})$ and the ν -optimal policy π_ν^* – for the largest class $\mathcal{M}_{\text{sol}}$ of all computable environments this is exactly as hard as computing π_ξ^* itself, but it may be much simpler for restricted classes, e.g. when $\mathcal{M}$ is a (sub)class of MDPs.

Theorem 9.2.1 & Corollary 9.2.2

Theorem 9.2.1 (Thompson sampling is asymptotically optimal in mean [Lei16b])

For all $\mu \in \mathcal{M}$, for the Thompson sampling agent π_{TS} , we have

$$\mathbb{E}_{\mu}^{\pi_{\text{TS}}} [V_{\mu}^*(h_{<t}) - V_{\mu}^{\pi_{\text{TS}}}(h_{<t})] \rightarrow 0 \quad \text{for } t \rightarrow \infty$$

Plain English. $\mathbb{E}$ denotes **expectation** (an average over randomness). This says that, *on average*, the gap between the best possible value V_{μ}^* and Thompson sampling's actual value shrinks to zero as time goes on – the agent is optimal “in mean”. This is weaker than being optimal on *every* run (Definition 8.1.7, strong asymptotic optimality) – indeed π_{TS} is *not* strongly asymptotically optimal, though it does converge in probability too.

Corollary 9.2.2: Sublinear Regret

Corollary 9.2.2 (Sublinear regret for Thompson sampling [Lei16b])

If the discount function γ satisfies Assumption 8.1.11, and the environment $\mu \in \mathcal{M}$ is recoverable, then the regret is sublinear: $\text{Regret}_m(\pi_{\text{TS}}, \mu) = o(m)$.

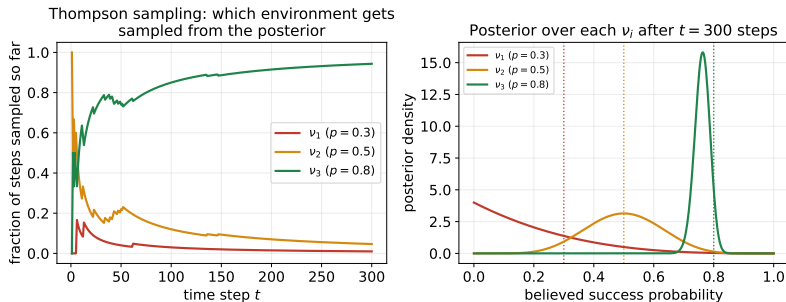
Plain English. Regret measures the total reward lost (over m steps) compared to always acting optimally. “Sublinear” means regret grows strictly slower than m itself, so the *average* per-step regret vanishes – a standard, strong notion of learning efficiency. **Recoverable** environments are ones where mistakes do not permanently destroy the chance of eventually acting well.

Python: Thompson Sampling on a Toy 3-Environment Class

We simulate Thompson sampling on a Bernoulli bandit with 3 candidate environments ν_1, ν_2, ν_3 with unknown success probabilities 0.3, 0.5, 0.8; the true environment is ν_3 . The posterior over each ν_i 's success probability is tracked with a Beta distribution (the standard conjugate posterior for Bernoulli observations).

```
1 import numpy as np
2 rng = np.random.default_rng(7)
3
4 true_p = np.array([0.30, 0.50, 0.80]) # nu_1, nu_2, nu_3
5 alpha, beta = np.ones(3), np.ones(3) # uniform Beta(1,1) priors
6
7 for t in range(300):
8     sample = rng.beta(alpha, beta) # draw one nu per candidate
9     a = int(np.argmax(sample)) # nu with highest sampled p
10    r = rng.binomial(1, true_p[a]) # follow pi*_nu, observe reward
11    alpha[a] += r; beta[a] += (1 - r) # Bayesian posterior update
12
13 print("posterior means:", alpha / (alpha + beta))
14 # -> posterior means: [0.29 0.45 0.76] (close to true_p, nu_3 favoured)
```

Thompson Sampling: Posterior Convergence



Left: the fraction of time steps in which each environment gets sampled converges to always picking the true best one, ν_3 . **Right:** after 300 steps, the posterior densities have sharpened around each environment's true success probability (dotted lines).

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents**
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 Summary

Motivation: What Should an Agent Without Rewards Want?

One of the fundamental questions in defining general intelligence is: where should rewards come from? Any reward provided by humans is susceptible to **manipulation** (discussed in Chapter 15) – and it is unclear to what degree an agent that simply maximizes externally supplied reward is autonomously intelligent at all. A reward function should not be tied to a specific goal (winning chess, driving from A to B), but should instead encourage *general* competence.

A reward function based purely on *how much the agent has learned* seems both general and useful. Agents built this way are called **curiosity-driven agents** [Sch91] or **Knowledge-Seeking Agents** (KSA). Seeking out information encourages exploration, which allows the agent to learn and solve problems.

We measure **Information Gain (IG)** using the **KL divergence** (Kullback–Leibler divergence, a standard measure of how different two probability distributions are; Definition 2.5.12) as a replacement for reward. Other distance measures exist, but most perform poorly in stochastic environments, chasing noise instead of genuine novelty [OLH13].

Definition 9.3.1: Information Gain

Definition 9.3.1 (Information Gain)

The one-step information gain of $h_k \in \mathcal{A} \times \mathcal{E}$ given history $h_{<k} \in (\mathcal{A} \times \mathcal{E})^{k-1}$ is defined as the KL divergence between the posterior conditioned on history $h_{1:k}$, and the posterior given history $h_{<k}$:

$$r_k^{\text{IG}} := \text{IG}(h_k | h_{<k}) := \sum_{\nu \in \mathcal{M}} w(\nu | h_{1:k}) \log \frac{w(\nu | h_{1:k})}{w(\nu | h_{<k})}$$

Plain English. $\mathcal{A} \times \mathcal{E}$ is the space of action-percept pairs; h_k is the single action-percept pair observed at time k . $\text{IG}(h_k | h_{<k})$ measures how much the posterior belief *changed* after seeing h_k : if h_k was very surprising (the posterior $w(\cdot | h_{<k})$ assigned it very low probability), the belief shifts a lot and the KL divergence – hence the information gain – is large. If the agent’s belief was already accurate ($w(\mu | h_{<k}) \approx 1$), it only observes what it expected anyway, so $w(\cdot | h_{1:k}) \approx w(\cdot | h_{<k})$ and the information gain is small.

This one-step definition extends naturally to an arbitrary (finite) history window $h_{k:k+\ell}$ instead of a single pair h_k ; we mainly use the one-step version, replacing the ordinary reward r_k in the usual value function (Definition 6.6.1) with the information-gain “reward” r_k^{IG} .

Definition 9.3.2: Information Gain Value Function

Definition 9.3.2 (Information Gain Value function)

The information gain value of policy π having observed history $h_{<t}$ with respect to discount function γ , denoted $V_{\text{IG}}^{\pi,m}$, is the ξ -expected discounted information gain:

$$V_{\text{IG}}^{\pi,m}(h_{<t}) := \mathbb{E}_{\xi}^{\pi} \left[\frac{1}{\Gamma_t} \sum_{k=t}^m \gamma_k \text{IG}(h_k | h_{<k}) \mid h_{<t} \right]$$

(we drop m when $m = \infty$). The **optimal information gain policy** π_{IG}^* maximizes this value function:

$$\pi_{\text{IG}}^*(\cdot) := \arg \max_{\pi} V_{\text{IG}}^{\pi}(\cdot) \quad \text{and} \quad V_{\text{IG}}^*(\cdot) := \max_{\pi} V_{\text{IG}}^{\pi}(\cdot)$$

Plain English. This is exactly the ordinary value function machinery from Chapter 6, but with the reward at each step replaced by the information gain at that step. The KSA policy π_{IG}^* is simply the policy that maximizes expected total (discounted) information gain – it seeks out actions likely to be the most surprising/informative, rather than the most rewarding.

Definition 9.3.3: KL-Divergence for ν^π and ξ^π

Information gain measures distance between successive *posteriors*. A related and useful quantity measures the distance between the **predictive policy-environment distributions** ν^π (what policy π predicts will happen if ν is the true environment) and ξ^π (what π predicts under the Bayes mixture).

Definition 9.3.3 (KL-Divergence for ν^π and ξ^π)

Given a history $h_{<k}$, the one-step KL divergence between ν^π and ξ^π is

$$\tilde{r}_k^\nu := \text{KL}_1(\nu^\pi, \xi^\pi | h_{<k}) := \sum_{h_k \in \mathcal{A} \times \mathcal{E}} \nu^\pi(h_k | h_{<k}) \log \frac{\nu^\pi(h_k | h_{<k})}{\xi^\pi(h_k | h_{<k})}$$

Plain English. Unlike r_k^{IG} , this quantity $\tilde{r}_k^\nu$ depends explicitly on ν and therefore *cannot* literally be used as a reward (the agent does not know the true ν) – but it is a valuable theoretical device, because (Equation 9.3.4) the corresponding ν -value function

$$\tilde{V}_{\nu, \gamma}^{\pi, m}(h_{<t}) \equiv \frac{1}{\Gamma_t} \mathbb{E}_\nu^\pi \left[\sum_{k=t}^m \gamma_k \tilde{r}_k^\nu \mid h_{<t} \right] \equiv \frac{1}{\Gamma_t} \sum_{k=t}^m \gamma_k \sum_{h_{t:k-1}} \nu^\pi(h_{t:k-1} | h_{<t}) \text{KL}_1(\nu^\pi, \xi^\pi | h_{<k})$$

lets us relate information gain (an agent-observable quantity) directly to the KL divergence between the true dynamics ν^π and the agent's believed dynamics ξ^π (a quantity about learning accuracy).

Lemma 9.3.5: Information Gain in Terms of KL Divergence

Lemma 9.3.5 (Information gain in terms of KL divergence)

The information gain value function can be expressed via the KL divergence between two policy-environment distributions:

$$V_{\text{IG}}^{\pi,m}(h_{<t}) = \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \tilde{V}_{\nu,\gamma}^{\pi,m}(h_{<t})$$

Plain English. This is the crucial bridge result: the information-gain value that the agent can actually compute and optimize (left-hand side) equals a posterior-weighted average, over every candidate environment ν , of how far the agent's beliefs currently are from that particular candidate's true dynamics (right-hand side). Chasing information gain is therefore mathematically the same as chasing accuracy about the (weighted) truth.

Proof outline. Expanding the definitions of V_{IG} and IG , pushing the sum over full histories $h_{t:m}$ inward so it cancels against the mixture normalization ($\sum_{h_{k:m}} \xi^\pi(h_{k:m}|h_{<t}) = 1$), rewriting $\nu(e_t|h_{<t}a_t)/\xi(e_t|h_{<t}a_t)$ using Definition 7.2.2 of the posterior, splitting $\nu^\pi(h_{t:k}|h_{<t})$ into a product over the step k , and finally recognizing Definition 9.3.3's KL divergence and substituting Equation 9.3.4, telescopes exactly into the claimed identity. ■

Theorem 9.3.6: On-Policy Prediction

Theorem 9.3.6 (On-policy prediction [OLH13])

Let $\mu \in \mathcal{M}$ and π be a policy, and $\sum_{\nu \in \mathcal{M}} w_{\nu} \log w_{\nu}^{-1} < \infty$, then

$$\mathbb{E}_{\mu}^{\pi} [\tilde{V}_{\mu}^{\pi}(h_{<t})] \xrightarrow{t \rightarrow \infty} 0$$

Plain English. The finiteness condition $\sum_{\nu} w_{\nu} \log w_{\nu}^{-1} < \infty$ says the prior's **entropy** is finite (roughly: the prior does not spread belief *too* thinly over too many hypotheses). Under that condition, this theorem shows that the mixture's on-policy predictions $P_{\xi}^{\pi}(\cdot|h_{<t})$ converge (in expectation, measured via cumulative discounted KL divergence) to the true predictions $P_{\mu}^{\pi}(\cdot|h_{<t})$. In other words, ξ becomes, in expectation, a good estimate of the unknown true predictive distribution – *as long as the agent keeps following policy π* (this is why it's called *on-policy*).

The finite-entropy condition fails for our usual canonical prior $w_{\nu}^U = 2^{-K(\nu)}$ on $\mathcal{M}_{\text{sol}}$ ($K(\nu)$ is the Kolmogorov complexity of ν – its shortest description length), but is easily satisfied by a marginally faster-decaying prior, e.g. $w_{\nu} = 2^{-(1+\varepsilon)K(\nu)}$, or $w_{\nu} = 2^{-K(\nu)}/K(\nu)$, or $w_{\nu_i} = 1/(i+1)^2$ (Exercise 7).

Proof of Theorem 9.3.6 I

The telescoping property of KL divergence (lifted to our setting, as in Lemma 3.2.4) gives

$$\begin{aligned}\mathbb{E}_\nu^\pi \left[\sum_{k=t}^m \text{KL}_1(\nu^\pi, \xi^\pi | h_{<t}) \right] &\equiv \mathbb{E}_\nu^\pi \left[\sum_{k=1}^m \text{KL}_1(\nu^\pi, \xi^\pi | h_{<t}) - \sum_{k=1}^{t-1} \text{KL}_1(\nu^\pi, \xi^\pi | h_{<t}) \right] \\ &= \text{KL}(\nu_{1:m}^\pi \| \xi_{1:m}^\pi) - \text{KL}(\nu_{<t}^\pi \| \xi_{<t}^\pi)\end{aligned}\tag{9.3.7}$$

As a sum of non-negative terms, $\text{KL}(\nu_{1:m}^\pi \| \xi_{1:m}^\pi)$ is increasing in m and converges to $c_\nu^\xi \leq \log w_\nu^{-1} < \infty$, by **dominance** $\xi^\pi \geq w_\nu \nu^\pi$ (Lemma 7.2.4: the mixture is always at least w_ν times any single constituent). Hence

$$\lim_{t \rightarrow \infty} \mathbb{E}_\nu^\pi \left[\sum_{k=t}^{\infty} \text{KL}_1(\nu, \xi | h_{<t}) \right] = c_\nu^\xi - c_\nu^\xi = 0.\tag{9.3.8}$$

Proof of Theorem 9.3.6 II

Also, using the positivity of KL divergence to introduce the missing sum over ν (a), and $\gamma_k/\Gamma_t \leq 1$ for $k \geq t$ together with the tower rule $\mathbb{E}[\mathbb{E}[\cdot|h_t]] = \mathbb{E}[\cdot]$ (b),

$$\mathbb{E}_\mu^\pi[\tilde{V}_\mu^\pi(h_{<t})] \stackrel{(a)}{\leq} \frac{1}{w_\mu} \sum_{\nu \in \mathcal{M}} w_\nu \mathbb{E}_\nu^\pi \left[\sum_{k=t}^{\infty} \tilde{V}_\nu^\pi(h_{<t}) \right] \stackrel{(b)}{\leq} \frac{1}{w_\mu} \sum_{\nu \in \mathcal{M}} w_\nu \mathbb{E}_\nu^\pi \left[\sum_{k=t}^{\infty} \tilde{r}_k^\nu \right] \quad (9.3.9)$$

Define $\delta_\nu(t) := \mathbb{E}_\nu^\pi \left[\sum_{k=t}^{\infty} \tilde{r}_k^\nu \right] \rightarrow 0$, which follows from Definition 9.3.3 and (9.3.8).

Proof of Theorem 9.3.6 III

We also know $\delta_\nu(t) \leq \text{KL}(\nu^\pi \| \xi^\pi) \leq \log w_\nu^{-1}$ from (9.3.7) and dominance. Unfortunately $\delta_\nu(t)$ is not *uniformly* bounded across ν , so we cannot directly conclude $\sum_\nu w_\nu \delta_\nu(t) \rightarrow 0$ (a lemma from [Hut05b] needs uniform boundedness). The fix: define $\tilde{w}_\nu := w_\nu \log w_\nu^{-1} > 0$ and $\tilde{\delta}_\nu(t) := \delta_\nu(t) / \log w_\nu^{-1} \leq 1$. By our finite-entropy assumption, $\sum_\nu \tilde{w}_\nu < \infty$, so now the lemma *does* apply to these “tilded” quantities, giving

$$0 \leq \mathbb{E}_\mu^\pi [\tilde{V}_\mu^\pi(h_{<t})] \leq \frac{1}{w_\mu} \sum_\nu \tilde{w}_\nu \tilde{\delta}_\nu(t) \xrightarrow{t \rightarrow \infty} 0. \quad \blacksquare$$

Theorem 9.3.10: On-Policy Learning, Off-Policy Prediction

The next result is perhaps the most important theoretical justification for defining π_{IG}^* at all. It shows that if the agent's history $h_{1:\infty}$ is generated by *following* π_{IG}^* , then $P_{\xi}^{\pi}(\cdot|h_{<t})$ converges in expectation to $P_{\mu}^{\pi}(\cdot|h_{<t})$ for *every* policy π – not just the one being followed. Informally: the longer the agent explores by chasing information, the better it can predict the counterfactual “what would happen if I followed policy π instead”.

Theorem 9.3.10 (On-policy learning, off-policy prediction [Ors11])

Let $\mu \in \mathcal{M}$ and $\sum_{\nu \in \mathcal{M}} w_{\nu} \log w_{\nu}^{-1} < \infty$. Then

$$\mathbb{E}_{\mu}^{\pi_{\text{IG}}^*} \left[\sup_{\pi} \tilde{V}_{\mu}^{\pi}(h_{<t}) \right] \xrightarrow{t \rightarrow \infty} 0$$

where the expectation is taken over $h_{<t}$.

Plain English. If the observation stream also included a reward signal, this off-policy prediction ability means the agent would asymptotically be able to *learn* (though not necessarily *follow*) the policy that maximizes expected discounted reward – the policy maximizing Bayes-expected reward would converge to optimal. Most policies only achieve the weaker on-policy guarantee of Theorem 9.3.6; achieving off-policy prediction is special to π_{IG}^* .

Proof of Theorem 9.3.10

By positivity of KL divergence (adding non-negative $\tilde{V}_\nu^\pi(h_{<t}) \geq 0$ terms for all ν), then Lemma 9.3.5 and the definition of π_{IG}^* as the maximizer of V_{IG} :

$$\begin{aligned}\mathbb{E}_\mu^{\pi_{\text{IG}}^*} \left[\sup_\pi \tilde{V}_\mu^\pi(h_{<t}) \right] &\leq \mathbb{E}_\mu^{\pi_{\text{IG}}^*} \left[\frac{1}{w(\mu|h_{<t})} \sup_\pi \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \tilde{V}_\nu^\pi(h_{<t}) \right] \\ &= \mathbb{E}_\mu^{\pi_{\text{IG}}^*} \left[\frac{1}{w(\mu|h_{<t})} \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \tilde{V}_\nu^{\pi_{\text{IG}}^*}(h_{<t}) \right]\end{aligned}$$

From the definition of the posterior and expectation, then exchanging the sum and expectation and using the definition of the posterior again:

$$= \frac{1}{w_\mu} \mathbb{E}_\xi^{\pi_{\text{IG}}^*} \left[\sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \tilde{V}_\nu^{\pi_{\text{IG}}^*}(h_{<t}) \right] = \frac{1}{w_\mu} \sum_{\nu \in \mathcal{M}} w_\nu \mathbb{E}_\nu^{\pi_{\text{IG}}^*} [\tilde{V}_\nu^{\pi_{\text{IG}}^*}(h_{<t})] \xrightarrow{t \rightarrow \infty} 0$$

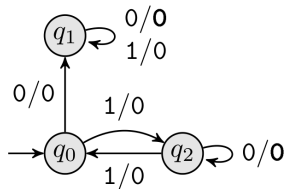
The limit follows from (9.3.9), choosing $\pi = \pi_{\text{IG}}^*$. ■

Knowledge-Seeking Agents: Takeaway

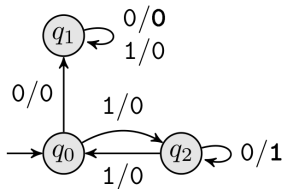
Takeaway. Knowledge-seeking agents aim to maximize value with respect to Bayesian mixtures over environments, and so share the same properties as Bayes-optimal agents: they depend on the choice of prior and can behave suboptimally in the short term. However, because KSAs continuously learn *off-policy* and are asymptotically optimal **without any recoverability assumption** (unlike Thompson sampling), the choice of prior ultimately does not affect long-run performance.

Example 9.3.11: KSA Experiments on Toy Environments

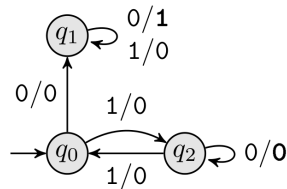
Consider the KL-KSA agent with a (deterministic) environment model class $\mathcal{M} = \{\mu_1, \mu_2, \mu_3\}$ given by the state diagrams below. All three environments are identical except for slight differences in the transitions highlighted in **bold**. State q_1 is a **trap state**: once visited, the agent can never leave.



(a) μ_1



(b) μ_2



(c) μ_3

From q_0 , action 1 moves to q_2 and back (obs. always 0); action 0 from q_0 moves permanently into the trap q_1 (obs. 0). The only place the three environments differ is what the self-loop observation is at q_1 under action 0 (environment μ_3 emits 1 instead of 0) and at q_2 under action 0 (environment μ_2 emits 1 instead of 0).

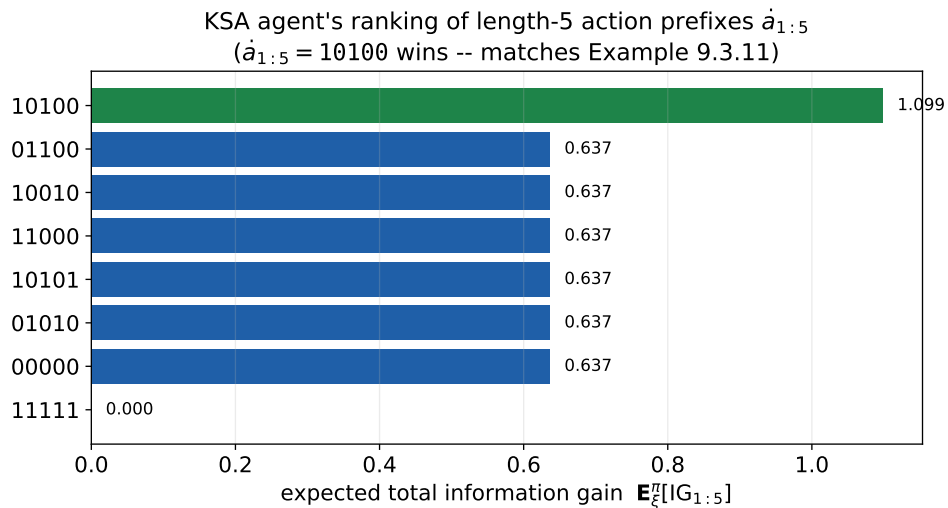
Example 9.3.11, Continued: The Optimal Exploration Sequence

To learn the true environment as fast as possible, the agent should take actions $\dot{a}_{1:5} = 10100$ and receive a sequence of observations $o'_{1:5}$ depending on the true environment: either 00000 for μ_1 , 01000 for μ_2 , or 00001 for μ_3 . **No other** action sequence can distinguish with certainty between the three environments in strictly fewer time steps.

Also, if the agent chooses $a_1 = 0$, it is stuck in the trap state forever, and can *never* distinguish μ_1 from μ_2 (if either is the true environment) – both produce identical observations at q_1 under action 0.

Of all possible action sequences $a_{1:\infty}$, the KSA agent values those prefixed with $\dot{a}_{1:5} = 10100$ the *highest*, indicating that it learns as fast as possible for this toy model class, and avoids moving into the trap state while there is still information to learn elsewhere.

Example 9.3.11: Ranking Action Prefixes by Information Gain



Python: Recreating Example 9.3.11 (1/2 – the Automaton)

We simulate the exact automaton from Figure 9.1: each candidate environment differs only in what it observes at the self-loops of q_1 and q_2 under action 0.

```
1  def step(state, action, env):           # env in {1,2,3}; Figure 9.1
2      obs_q1_a0 = {1: 0, 2: 0, 3: 1}[env] # self-loop obs. at trap q1
3      obs_q2_a0 = {1: 0, 2: 1, 3: 0}[env] # self-loop obs. at q2
4      if state == "q0":
5          return ("q1", 0) if action == 0 else ("q2", 0)
6      if state == "q1":
7          return ("q1", obs_q1_a0 if action == 0 else 0)
8      return ("q2", obs_q2_a0) if action == 0 else ("q0", 0)
9
10 def obs_sequence(actions, env):
11     s, out = "q0", []
12     for a in actions:
13         s, o = step(s, a, env)
14         out.append(o)
15     return tuple(out)
```

Python: Recreating Example 9.3.11 (2/2 – Information Gain)

Now compute the expected total information gain, Definitions 9.3.1–9.3.2 (undiscounted, over 5 steps, uniform prior $\frac{1}{3}$ over μ_1, μ_2, μ_3), for several candidate 5-action prefixes.

```
1 def expected_info_gain(actions, envs=(1, 2, 3)):
2     obs = {e: obs_sequence(actions, e) for e in envs}
3     total = 0.0
4     for true_env in envs:                                     # average over which env is true
5         w = {e: 1/3 for e in envs}
6         for k in range(len(actions)):
7             w_new = {e: (w[e] if obs[e][:k+1] == obs[true_env][:k+1] else 0.0)
8                       for e in envs}
9             z = sum(w_new.values()); w_new = {e: v/z for e, v in w_new.items()}
10            for e in envs:
11                if w_new[e] > 0 and w[e] > 0:
12                    total += (1/3) * w_new[e] * __import__("math").log(w_new[e]/w[e])
13            w = w_new
14    return total
15
16 for a in ["10100", "00000", "11111"]:
17     print(a, "->", round(expected_info_gain([int(c) for c in a]), 3))
18 # 10100 -> 1.099      (= log 3: fully distinguishes all three environments)
19 # 00000 -> 0.637      (stuck in trap; only rules mu_3 in/out, never mu_1 vs mu_2)
20 # 11111 -> 0.0        (never takes action 0; observations identical for all env)
```

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)**
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 Summary

The Exploration–Exploitation Problem

While acting in an environment, we often want the agent to explore enough that an optimal action can eventually be identified. How much time should the agent spend exploring? Exploring too much means the agent never acts optimally, only ever taking novel actions. Exploring too little means the agent prematurely commits to an action based on an inaccurate model. This trade-off is the classic **exploration–exploitation problem**.

Unexpectedly, the Bayes-optimal agent $AI\xi$ does *not* explore enough to be asymptotically optimal (Theorem 8.3.1) – but this can be fixed by adding “a bit” of extra, deliberate exploration, though it is not obvious *whether* we should always want to [CHC21].

Algorithm 9.4: BayesExp

Under reasonable assumptions, there exists an agent that balances exploration and exploitation to achieve (weak) asymptotic optimality: **BayesExp**, a Bayesian agent with extra exploration, switches between exploring and exploiting in just the right way that asymptotically the exploiting phase becomes optimal – though there is always a non-zero chance of exploring.

Require: Non-increasing sequence $\varepsilon_1, \varepsilon_2, \varepsilon_3, \dots$

Require: Model class $\mathcal{M}$. Prior w . Effective horizon function $H_t(\cdot)$.

Input: Percept stream $e_1, e_2, e_3, \dots$

Output: Action stream $a_1, a_2, a_3, \dots$

```
1: while True do
2:   if  $V_{IG}^{*, t+H_t(\varepsilon_t)}(h_{<t}) > \varepsilon_t$  then
3:     follow  $\pi_{IG}^*$  for  $H_t(\varepsilon_t)$  steps ▷ exploration
4:   else
5:     follow  $\pi_{\xi}^*$  for 1 step ▷ exploitation
6:   end if
7: end while
```

Algorithm 9.4: Plain-English Summary

Plain English. At each decision point, BayesExp checks whether the *best achievable* expected information gain over the next $H_t(\varepsilon_t)$ steps exceeds the current threshold ε_t . If yes, it commits to exploring (following the knowledge-seeking policy π_{IG}^*) for that whole block of steps. If not, it takes a single Bayes-optimal exploitation step and re-checks.

Theorem 9.4.1: BayesExp Is Weakly Asymptotically Optimal

Theorem 9.4.1 (BayesExp is weakly asymptotically optimal [Lat14])

Let $\mathcal{M} = \{\nu_1, \nu_2, \dots\}$ be countable and $\sum_{\nu \in \mathcal{M}} w_\nu \log w_\nu^{-1} < \infty$, and discount γ_t and exploration thresholds ε_t satisfying

- $\Gamma_t > 0$ for all t
- $H_{t+1}(\varepsilon) \geq H_t(\varepsilon)$, for all $\varepsilon > 0$ and all $t \in \mathbb{N}$
- $\varepsilon_{t+1} \leq \varepsilon_t$ for all t
- $\lim_{t \rightarrow \infty} \varepsilon_t = 0$
- $H_t(\varepsilon_t) = o(t\varepsilon_t)$

Then BayesExp is weakly asymptotically optimal in $(\mathcal{M}, \gamma)$.

Plain English. These conditions just say: the effective horizon doesn't shrink over time, the exploration threshold decreases (but not instantly to zero), and the total time spent exploring ($H_t(\varepsilon_t)$) grows strictly slower than $t\varepsilon_t$. Given all that, BayesExp achieves **weak asymptotic optimality** (Definition 8.1.5: optimal on average / in the limit of Cesàro averages).

Proof Idea for Theorem 9.4.1

Proof idea [Lat14]. (i) π_ξ^* converges to the truly optimal π_μ^* on BayesExp's exploitation steps; (ii) the number of exploration steps taken is $o(n)$ (a vanishing fraction of total time); (iii) the value of π_{BE} converges to the value of π_ξ^* in Cesàro average. Together these imply weak asymptotic optimality. ■

Why BayesExp Works

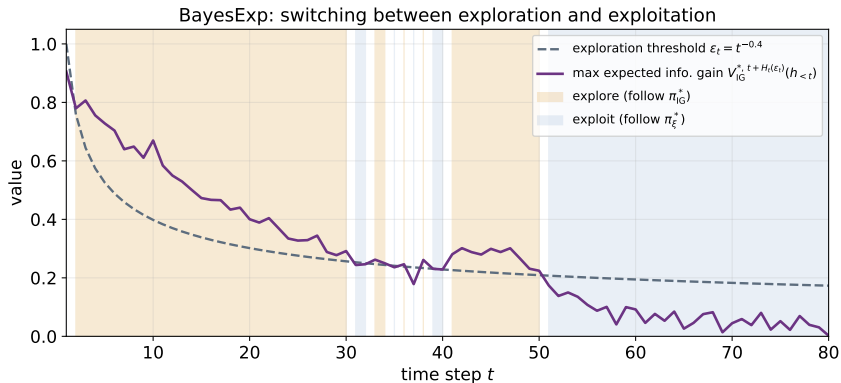
BayesExp's ability to balance exploration and exploitation stems from its *dynamic* exploration strategy: it adjusts the number of exploration steps based on the maximum expected information gain and a predefined threshold.

- As the agent progresses, the threshold ε_t decreases toward 0, ensuring BayesExp keeps exploring *as long as* information can still be gained.
- At the same time, $\varepsilon_t \rightarrow 0$ *slowly enough* that the probability of exploration also tends to 0 overall – this gradual transition is what makes the trade-off work.
- One can show that for *any* discount with a monotonically but sublinearly growing effective horizon, a valid exploration sequence ε_t exists (Exercise 4); e.g. geometric discount $\gamma_t = \gamma^t$ has constant effective horizon, and $\varepsilon_t = t^{-\beta}$ (β (beta), any $0 < \beta < 1$) works.

Why BayesExp Works, Continued

- For deterministic environments and geometric discounting, the proof is much simpler (environments can be ruled out completely, as in Section 9.1); the stochastic-case proof is beyond the scope of the book [Lat14].
- Since $\varepsilon_t \rightarrow 0$, at exploitation time-steps the expected information gain converges to 0, meaning ξ concentrates around μ while exploiting – so BayesExp is close-to-optimal with probability tending to 1 while exploiting.
- Exploring in *blocks* (rather than single steps) avoids problems with **time-inconsistency** (Section 6.5): the exploration policy depends on the horizon, so recomputing it every step could chain together different policies in a way that leads to poor behaviour. Committing to π_{IG}^* for $H_t(\varepsilon_t)$ steps at a time avoids this.

BayesExp: Explore/Exploit Timeline



The Inquisitive Agent (Inq)

The **Inquisitive Reinforcement Learner (Inq)** achieves **strong** asymptotic optimality (Definition 8.1.7) – optimal on *every* run, not just on average. Similar to BayesExp, Inq combines Bayes-optimal $AI\xi$ with the knowledge-seeking agent. But instead of prescribing exploration blocks deterministically, Inq follows maximally informative exploratory “expeditions” with a certain *probability*; unlike BayesExp, these expeditions have *random* length, possibly shorter or longer than the effective horizon.

Also, Inq uses an *undiscounted*, multi-step version of Definition 9.3.1, and the effective horizon must be **bounded**. Otherwise Inq follows the same principle as BayesExp: it is more likely to explore the more it expects to reduce its uncertainty about which environment it is in. As certainty about the true environment μ grows, exploration becomes rarer, and asymptotically Inq converges to $AI\xi$. The additional exploration prevents Inq from getting stuck ignorant about unexplored parts of the world, which guarantees it also converges to $AI\mu$ (the Bayes-optimal agent if μ were known).

Theorem 9.4.2 (Inq is strongly asymptotically optimal [CCH19])

For any countable class of environments $\mathcal{M}$ and bounded effective horizon ($\sup_t H_t(\varepsilon) < \infty \forall \varepsilon > 0$), the Inquisitive Reinforcement Learner (Inq, Algorithm 9.5) is strongly asymptotically optimal.

Building Inq: Multi-Step Information Gain I

First we need a multi-step generalization of the one-step ($t = n$) information gain from Definition 9.3.1:

$$\text{IG}_{t:n}(h_{t:n}|h_{<t}) := \sum_{\nu \in \mathcal{M}} w(\nu|h_{1:n}) \log \frac{w(\nu|h_{1:n})}{w(\nu|h_{<t})}$$

This leads to the non-discounted truncated Information Gain Value function for policy π :

$$V_{\text{IG}}^{\pi,n}(h_{<t}) := \mathbb{E}_{\xi}^{\pi}[\text{IG}_{t:n}|h_{<t}] \equiv \sum_{h_{t:n}} \xi^{\pi}(h_{t:n}|h_{<t}) \text{IG}_{t:n}(h_{t:n}|h_{<t})$$

and similarly to the knowledge-seeking agent, the optimal (deterministic) IG policy for this window is

$$\pi_{\text{IG},h_{<t}}^{*,n} := \arg \max_{\pi} V_{\text{IG}}^{\pi,n}(h_{<t})$$

This policy is well-defined on histories from length $t - 1$ up to n .

Building Inq: Multi-Step Information Gain II

At each time step t , the action taken is $a_t := \pi_{\text{IG}, h_{<t-k}}^{*, t-k+m-1}(h_{<t})$, for an IG policy that was optimized at some *random* k time steps earlier, with a *random* horizon m . While the IG policy is (re-)sampled independently at each t , there is a non-zero chance of following the *same* policy for $H_t(\varepsilon)$ time steps in a row, since $m > H_t(\varepsilon) - k$ may be sampled, and the policy that started at t may be sampled again for another $H_t(\varepsilon)$ steps. Exploring consistently for $H_t(\varepsilon)$ steps at a time matters – BayesExp *prescribes* this, Inq achieves it only with non-zero probability.

The probability of sampling $\pi_{\text{IG}, h_{<t-k}}^{*, t-k+m-1}$ is

$$\varepsilon_k^m(h_{<t}) := \min \left\{ \frac{1}{m^2(m+1)}, \quad \eta V_{\text{IG}}^{\pi_{\text{IG}, h_{<t-k}}^{*, t-k+m-1}, t+m}(h_{<t}) \right\}$$

for $m \in \mathbb{N}^+$ and $0 \leq k < \min\{m, t\}$, where $0 < \eta < 1$ (eta) is an exploration constant.

Building Inq: Multi-Step Information Gain III

The first term, $1/(m^2(m+1))$, ensures the probabilities never sum to more than 1: it caps how much probability mass is spent exploring for any particular (k, m) pair, and

$$\varepsilon_+(h_{<t}) := \sum_{m \in \mathbb{N}^+} \sum_{k < \min\{m, t\}} \varepsilon_k^m(h_{<t}) \leq \sum_{m \in \mathbb{N}^+} \sum_{k < m} \frac{1}{m^2(m+1)} = 1.$$

The remaining probability, $1 - \varepsilon_+(h_{<t})$, is used for following the Bayes-optimal policy π_ξ^* . Together these define the Inq algorithm.

Algorithm 9.5: Inq

Require: Model class $\mathcal{M}$. Prior w . Discount γ .

Input: Percept stream $e_1, e_2, e_3, \dots$

Output: Action stream $a_1, a_2, a_3, \dots$

- 1: **for** $t = 1, 2, 3, \dots$ **do**
- 2: Take action $\pi_{\text{IG}, h_{<t-k}}^{*, t-k+m-1}(h_{<t})$ with probability $\varepsilon_k^m(h_{<t})$ ▷ exploration
- 3: Take action $\pi_{\xi}^*(h_{<t})$ with probability $1 - \varepsilon_+(h_{<t})$ ▷ exploitation
- 4: **end for**

Plain English. At every single time step, Inq flips a weighted coin: with the small probabilities ε_k^m (summed over all valid (k, m)) it continues (or starts) an exploratory expedition guided by an information-gain-optimal policy computed for some window; otherwise (the leftover probability $1 - \varepsilon_+$) it simply exploits via the Bayes-optimal action.

Contrast with BayesExp. Strong asymptotic optimality may not be achievable for environments with (some or all) *unbounded* horizon, since the horizon may grow faster than Inq – or possibly any policy – can learn about ever-longer-term dynamics. BayesExp, by contrast, is *weakly* asymptotically optimal even for (sublinearly) growing horizons. Both agents maximize information gain (explore) whenever enough information about the environment is expected to be gained; if not, they follow the Bayes-optimal agent (though calling π_ξ^* “exploiting” is not entirely fair – $\text{AI}\xi$ implicitly explores too, just not *enough* to be asymptotically optimal on its own).

Python: Simulating BayesExp's Explore/Exploit Switch

A minimal simulation of BayesExp's decision rule, using a synthetic (illustrative) trace of the maximum expected information gain V_{IG}^* and the threshold schedule $\varepsilon_t = t^{-\beta}$.

```
1 import numpy as np
2 T, beta = 20, 0.4
3 t = np.arange(1, T + 1)
4 eps_t = t ** (-beta)           # decreasing exploration threshold
5
6 # Toy V_IG*: starts high (lots to learn), decays as the agent learns
7 rng = np.random.default_rng(1)
8 V_IG = 0.9 * np.exp(-t / 8.0) + rng.normal(0, 0.02, T)
9
10 mode = np.where(V_IG > eps_t, "EXPLORE", "exploit")
11 for i in range(T):
12     print(f"t={t[i]:2d}  V_IG*={V_IG[i]:.3f}  eps_t={eps_t[i]:.3f}  -> {mode[i]}")
13 # t= 1  V_IG*=0.812  eps_t=1.000  -> exploit
14 # t= 2  V_IG*=0.727  eps_t=0.758  -> exploit
15 # t= 3  V_IG*=0.634  eps_t=0.644  -> exploit
16 # t= 4  V_IG*=0.577  eps_t=0.574  -> EXPLORE
17 # ...
18 # t=20  V_IG*=0.083  eps_t=0.312  -> exploit
```

Observation. Early on, when there is a lot of information to be gained, the agent explores; as V_{IG}^* decays below the shrinking threshold ε_t , BayesExp settles into exploiting the Bayes-optimal policy π_{ξ}^* – matching the timeline figure on the previous slide.

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)**
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 Summary

Avoiding Expensive Planning

Rather than explicit **expectimax planning** (which in practice is typically approximated by Monte Carlo Tree Search), there exist reinforcement-learning algorithms that side-step planning entirely by learning a **Q-value function** directly – via solving Bellman (optimality) equations in theory, or via **Temporal Difference (TD)**-style algorithms in practice – or by directly learning a policy.

Planning-avoiding universal agents have also been developed. In this section we introduce **Self-AIXI** [CGMH+23], which self-predicts its own action stream. (Two related approaches: *AIXItl*, Section 13.2, computes a provable lower bound to the value function; *Compress and Control*, Section 12.9, is another way to avoid planning.)

Self-AIXI is a universal agent using **self-prediction** instead of traditional planning. It generates a stream of action data akin to methods used by other TD(0) agents, executing a one-step action maximization based on current *on-policy* Q-value estimates for a mixture-policy. Despite only performing one-step-lookahead “planning”, Self-AIXI converges to the (long-horizon) Bayes-optimal $AI\xi$, is self-optimizing, and has maximal Legg–Hutter intelligence.

Definition 9.5.1: Bayesian Mixture Policy and Self-AIXI

Self-AIXI predicts its own stream of actions via a Bayesian mixture policy ζ (zeta) over a class of policies Π , in a dual fashion to the Bayesian mixture *environment* ξ over $\mathcal{M}$:

Definition 9.5.1 (Bayesian mixture policy and Self-AIXI)

The Bayesian mixture ζ over a countable class of policies Π is defined as

$$\zeta(a_t|h_{<t}) := \sum_{\pi \in \Pi} \omega(\pi|h_{<t}) \pi(a_t|h_{<t}), \quad \omega(\pi|h_{1:t}) := \omega(\pi|h_{<t}) \frac{\pi(a_t|h_{<t})}{\zeta(a_t|h_{<t})} = \omega_\pi \frac{\pi(a_{1:t}|e_{<t})}{\zeta(a_{1:t}|e_{<t})}$$

Self-AIXI takes the one-step optimal action according to *both* the Bayesian mixture environment ξ over $\mathcal{M}$ with prior w_ν *and* the Bayesian mixture policy ζ over Π with prior $\omega(\pi|\epsilon) \equiv \omega_\pi$:

$$\pi_S(h_{<t}) := \arg \max_{a_t} Q_\xi^\zeta(h_{<t}a_t)$$

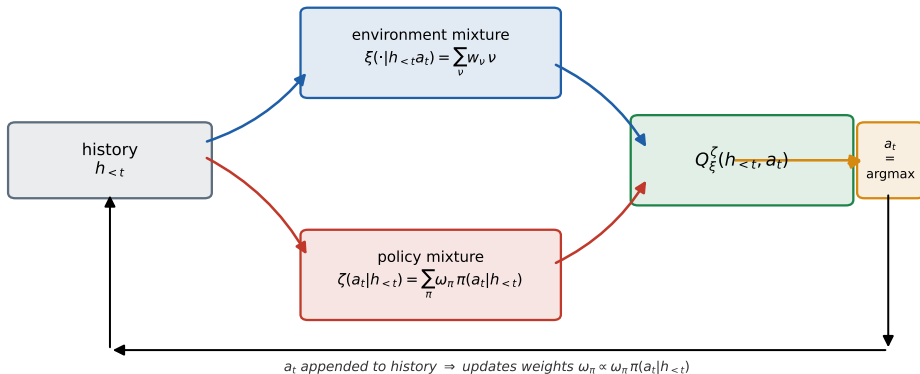
where the Q-value is as in Definition 6.7.1.

Definition 9.5.1: Plain-English Summary

Plain English. ω_π (omega) is a prior weight over *policies*, exactly analogous to w_ν over environments. ζ is a mixture – a weighted blend – of many candidate policies, updated by Bayes' rule just like ξ is updated over environments (each observed action a_t reweights $\omega(\pi|h_{1:t})$ toward policies that would have predicted it).

Understanding Self-AIXI's Action Rule

Self-AIXI: two Bayesian mixtures (over environments and over policies)
jointly determine the one-step-optimal action



Self-AIXI: The Self-Improving Loop

Importantly, Self-AIXI maximizes the **Q-values from the mixture policy** ζ instead of the fully optimal policy π_ξ^* . This means there is no need to optimize over the (expensive, long-horizon) future: only the *on-policy* Q-values of the current ζ are needed, which are typically far easier to estimate than the optimal off-policy Q-values Q_ξ^* .

Self-improving loop. The action selected by Self-AIXI necessarily improves, by definition of $\arg \max$, over the current value estimate: $\max_{a_t} Q_\xi^\zeta(h_{<t}a_t) \geq V_\xi^\zeta(h_{<t})$. This action is then appended to the next history $h_{<t+1}$, which is in turn consumed by the policy-mixture ζ at the next time step – so ζ does Bayesian inference over its own, self-generated stream of actions, and makes progressively better action predictions over time. Self-AIXI is thus *self-predicting* its own small, incremental improvements.

Theorem 9.5.2: Convergence Properties of Self-AIXI

In the case of using the largest class of *all* computable policies, the mixture ζ becomes a Solomonoff (action) predictor with enough representational power to approximate, within itself, both the policy evaluation *and* the policy improvement operation.

Theorem 9.5.2 (Convergence properties of Self-AIXI [CGMH+23])

Under various conditions, the value functions of Self-AIXI π_S converge to those of Bayes-optimal $\text{AI}\xi$ in expectation on histories generated by π_S interacting with μ :

$$\mathbb{E}_{\mu}^{\pi_S} [V_{\xi}^{\pi_{\xi}^*}(h_{<t}) - V_{\xi}^{\pi_S}(h_{<t})] \xrightarrow{t \rightarrow \infty} 0, \quad \mathbb{E}_{\mu}^{\pi_S} [V_{\mu}^{\pi_{\mu}^*}(h_{<t}) - V_{\mu}^{\pi_S}(h_{<t})] \xrightarrow{t \rightarrow \infty} 0$$

Plain English. Both the value *Self-AIXI believes it is getting* (w.r.t. ξ) and the value *it actually gets* (w.r.t. the true μ) converge to the Bayes-optimal value – i.e. despite only ever doing one-step lookahead, Self-AIXI eventually performs as well as full long-horizon Bayes-optimal planning.

Consequences of Theorem 9.5.2

This implies that for environment classes $\mathcal{M}$ admitting **self-optimizing** policies (Figure 7.1), since $\text{AI}\xi$ is self-optimizing (Theorem 7.3.6), **Self-AIXI is too**:

$$V_{\mu}^*(h_{<t}) - V_{\mu}^{\pi_S}(h_{<t}) \xrightarrow{t \rightarrow \infty} 0 \quad \mu^{\pi}\text{-almost surely, for any } \pi$$

For $\pi = \pi_S$, this implies π_S is **strongly asymptotically optimal**. For $\mathcal{M} = \mathcal{M}_{\text{sol}}$ (the class of all computable environments), the first convergence result implies Self-AIXI asymptotically has **maximal universal intelligence** Υ (Upsilon, the Legg–Hutter intelligence measure, Definition 16.7.1):

$$\mathbb{E}_{\mu}^{\pi_S} \left[\max_{\pi} \Upsilon(\pi|h_{<t}) - \Upsilon(\pi_S|h_{<t}) \right] \xrightarrow{t \rightarrow \infty} 0$$

Proof idea. Self-AIXI initially acts like a one-step optimal (greedy) agent, and this shows up in the action sequence it produces. The mixture ζ then *learns* to act like a one-step optimal agent, which means Self-AIXI starts acting like a *two*-step optimal agent. This bootstrapping continues indefinitely, and the performance of Self-AIXI converges to that of $\text{AI}\xi$. ■

These results show that agents can perform well in the general reinforcement learning setting *without* utilizing explicit planning at all. Experiments comparing Self-AIXI with MC-AIXI-CTW (Chapter 12) and full proofs are in [CGMH+23].

Python: A Toy Self-Prediction Loop

A stripped-down illustration of the self-improving loop: two candidate policies (mixture components) compete for weight ω_π as Self-AIXI's own actions are fed back in as data.

```
1 import numpy as np
2 policies = {                                # toy:  $P(\text{action}=1 \mid h)$  for two policies
3     "explore_biased": 0.7,
4     "exploit_biased": 0.2,
5 }
6 omega = {"explore_biased": 0.5, "exploit_biased": 0.5}    # prior  $\omega_{\pi_i}$ 
7
8 def zeta(a):                                            # mixture policy  $\zeta(a_t \mid h_{<t})$ 
9     return sum(omega[p] * (policies[p] if a == 1 else 1 - policies[p])
10               for p in policies)
11
12 rng = np.random.default_rng(0)
13 for t in range(6):
14     # Self-AIXI would take  $a_t = \operatorname{argmax}_a Q^{\zeta_{\text{xi}}}(h, a)$ ; here we just
15     # illustrate the *belief update* step after an action is produced.
16     a_t = 1 if rng.random() < zeta(1) else 0
17     for p in policies:
18         like = policies[p] if a_t == 1 else 1 - policies[p]
19         omega[p] *= like
20     z = sum(omega.values()); omega = {p: v / z for p, v in omega.items()}
21     e1, e2 = omega["explore_biased"], omega["exploit_biased"]
22     print("t=", t, " a_t=", a_t, " omega_explore=", round(e1, 3), " omega_exploit=", round(e2,
```


Self-Prediction Loop: Idea

Idea. Every self-generated action a_t is Bayesian evidence about which policy in the mixture best explains Self-AIXI's own past behaviour, exactly as Definition 9.5.1's update rule for $\omega(\pi|h_{1:t})$ prescribes.

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises**
- 8 9.7 History and References
- 9 Summary

Exercises I

1. [C25] Prove Theorem 9.1.3.
2. [C32] Prove Theorem 9.2.1.
3. [C28] Prove Theorem 9.4.1.
4. [C20] Prove that if $H_t(\varepsilon) = o(t)$, then there exists $\varepsilon_t \rightarrow 0$ such that $H_t(\varepsilon_t) = o(t\varepsilon_t)$ as required in Theorem 9.4.1.
5. [C45o] Prove/disprove that Thompson sampling is weakly asymptotically optimal.
6. [C45o] Prove/disprove that using Thompson sampling with an ε -optimal policy for the sampled environment leads to an agent which is asymptotically optimal in expectation.
7. [C20i] Prove that $w_\nu = 2^{-K(\nu)}$ has infinite entropy $\sum_{\nu \in \mathcal{M}_{\text{sol}}} w_\nu \log w_\nu^{-1}$, but $w_\nu = 2^{-(1+\varepsilon)K(\nu)}$ and $w_\nu = 2^{-K(\nu)}/K(\nu)$ have finite entropy.
8. [C38] BayesExp uses explicit exploration to achieve weak asymptotic optimality. Bayesian agents like AIXI will still (implicitly) explore. Derive a definition for the kind of exploration a Bayesian agent does. Using this definition, find the largest class of environments $\mathcal{M}$ such that AIXI is asymptotically optimal (for each type).

9. [C45o] Prove/disprove that using BayesExp with an ε -optimal policy during the exploitation (and/or exploration) phases leads to an agent which is weakly asymptotically optimal?

Reading the difficulty codes. The bracketed code (e.g. [C25], [C45o]) is the book's difficulty rating: roughly, the number gives an estimated difficulty out of 50, "C" marks a *creative*/research-flavoured exercise, and a trailing "o" marks an *open* problem without a currently known complete solution.

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References**
- 9 Summary

Optimism. Optimism has long been an approach to the exploration problem in reinforcement learning [SL08]. The optimistic variant of AIXI and its philosophical and axiomatic underpinning were first introduced in [SH12b], where it was shown how optimism can lead to asymptotic optimality (Section 9.1). Optimism of general agents was further studied in the context of cognitive theory and rational behaviour [SH14a,SH15b]. On the opposite end of the spectrum, pessimism in general reinforcement learning can lead to more *safe* behaviour [CH20b]. A counter-point to optimism was presented in [OVR16b], which argued that any optimistic algorithm matching Thompson sampling in statistical efficiency would likely be computationally intractable.

History and References II

Thompson sampling. Thompson sampling [Tho33] (Section 9.2) has had a recent resurgence, showing strong performance in many problems beyond the traditional bandit setting, including: contextual bandits [AG13], finite-time bandits [CPRR17], non-stationary bandits [RK17], MDPs [GM15, OGNJ17, OVR16a], POMDPs [JJJN21], game theory [Mau20], adaptive control [OB10b, OB10a, Ort11], and (general) reinforcement learning [LLOH16, LLOH17]. A thorough overview is given in [RVRK+17]. Using naive sequential prediction to solve sequential decision-making problems can lead to several difficulties [OKD+21]; however [OWR+19] showed how to *meta-learn* a Thompson sampling agent and overcome many of these difficulties. Thompson-sampling-based exploration has been shown to work well in Atari when using an uncertainty-based Bellman equation [OOMM17].

History and References III

Knowledge-seeking agents. The knowledge-seeking agent was first proposed in [Ors11] and later extended to stochastic environments in [OLH13] (Section 9.3). It was extended again from stochastic to *quantum* environments in [Sar21,SAAGB21].

Curiosity-based agents. Highly related to knowledge-seeking agents are curiosity-based agents: agents that use a notion of curiosity to aid exploration, or in some cases replace reward entirely. One of the earliest curiosity-based agents was [Sch91], incentivized to take actions providing more information about the environment. [BEP+18] studied curiosity-based agents in Atari and other RL environments. A known weakness of curiosity/information-seeking agents is susceptibility to “noisy TVs”: any input feeding unpredictable random data to the agent looks endlessly informative. The original knowledge-seeking agent [Ors11] (which only modelled deterministic environments) suffers this problem. [MPYBG22] investigated how curiosity-based agents can overcome it. Variational Information Maximizing has been shown to be an effective curiosity-based exploration method [HCD+16]. [SGS11] introduced an exploration strategy using a curiosity-based (action) value function and proved its (Bayesian exploration) optimality when the environment is an MDP. Curiosity is not always positive for an agent, and the dangers of *overly* curious agents were discussed in [CHC21].

Intrinsic rewards. One key aspect of knowledge-seeking agents is that they possess an **intrinsic reward**. Intrinsic motivation and reward has seen substantial study; [LAW+20] surveys many approaches. [BSC+04] studied intrinsic motivation's ability to lead to hierarchical learning. [Sch10] proposed a precise theory of creativity, fun, and intrinsic motivation, itself motivated by compression. One approach in the absence of rewards is to use the discovery of new behaviour in the form of *options* [SP02] to motivate the agent [MB16]. Intrinsic motivation can also guide the exploration of agents [BSO+16,MBMK21]. This topic will see further exploration in Chapter 15.

History and References V

General exploration agents. The BayesExp agent [LH14a] (Section 9.4) was the first agent shown to be weakly asymptotically optimal in a large class of environments, demonstrating that weak asymptotic optimality was an *achievable* property of general agents (though not deterministic ones). [Lei16a] introduced an exploration measure, called *exploration potential*, provably equivalent to asymptotic optimality (under some conditions). The Inq agent [CCH19] was the first agent shown to be *strongly* asymptotically optimal, demonstrating (with BayesExp) that strong asymptotic optimality was an achievable property – previously suspected, but not proven, that no general agent could achieve this.

Planning-avoiding agents. For finite-state MDPs, Bellman equations [BT96] and Temporal Difference algorithms [HL07,SB18] bootstrap values and policies by one-step (or few-step) lookahead instead of expensive (long-horizon) expectimax or Monte Carlo Tree Search planning. These ideas extend to infinite MDPs via (non)linear value function approximation and POMDPs. In this chapter we discussed the history-based planning-avoiding agent Self-AIXI [CGMH+23], which maintains a Bayesian posterior mixture over policies. Section 13.2 discusses AIXIt/ [Hut05b, Chp. 7], and Section 12.9 discusses Compress and Control [VBH+15,DRW+24].

History and References VI

Risk- and ambiguity-sensitivity agents. Bayes-optimal agents are **risk-neutral** since they attune solely to the expected return, in contrast to **risk-sensitive** agents, which are also sensitive to higher-order moments of the return. Bayes-optimal agents are also **ambiguity-neutral** since they act in new situations as if the uncertainty were known, unlike **ambiguity-sensitive** agents, which act differently when recognizing situations in which they lack knowledge. [GMDK+22] show how off-policy meta-learning can give rise to risk- and ambiguity-sensitive agents, which can be safer and more robust to external perturbations.

Outline

- 1 Introduction: Why Other Agents?
- 2 9.1 Optimistic Agents
- 3 9.2 (Thompson) Sampling Agents
- 4 9.3 Knowledge-Seeking Agents
- 5 9.4 Exploring Agents (BayesExp and Inq)
- 6 9.5 Planning-Avoiding Agents (Self-AIXI)
- 7 9.6 Exercises
- 8 9.7 History and References
- 9 **Summary**

Chapter 9 Summary

- **Optimistic agents** (π°) replace Bayesian averaging with a best-case maximization over the still-consistent environment class $\mathcal{M}_t$; provably asymptotically optimal (Theorem 9.1.1) with an explicit finite error bound (Theorem 9.1.2), extending from finite deterministic classes to stochastic, countable, compact, and separable classes.
- **Thompson sampling** (π_{TS}) samples one environment from the posterior and commits to its optimal policy for a while; asymptotically optimal *in mean* (Theorem 9.2.1) with sublinear regret (Corollary 9.2.2), though not strongly asymptotically optimal.
- **Knowledge-seeking agents** (π_{IG}^*) replace reward with information gain (KL divergence between successive posteriors); they learn on-policy (Theorem 9.3.6) *and* off-policy (Theorem 9.3.10), making them immune to poor prior choices in the long run.
- **BayesExp and Inq** add deliberate exploration on top of $\text{AI}\xi$: BayesExp explores in scheduled blocks (weakly asymptotically optimal, Theorem 9.4.1); Inq explores randomly with history-dependent probability (strongly asymptotically optimal for bounded horizons, Theorem 9.4.2).
- **Self-AIXI** (π_{S}) avoids expensive planning by maximizing one-step Q-values under a self-predicting Bayesian mixture *policy* ζ , dual to the mixture *environment* ξ ; it converges to Bayes-optimal $\text{AI}\xi$ (Theorem 9.5.2) and inherits maximal Legg–Hutter intelligence.

Next: Chapter 10 turns to computable approximations of these universal agents.